

SD Dictionaries - Conversions and Formatting

Every conversion code SD recognises, and what each one does to a stored value.

A dictionary record does two things: it says where to find the data (field 2), and it says how to display it (fields 3 and 5). Field 3 is the **conversion code** — a compact instruction that transforms a stored value into a display value. This page is every conversion code SD recognises; field 5, the format specification, is on the page that continues it.

SD folds case, so a conversion code may be typed in either case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every conversion code on this page was read from the SD source on SD Core for Windows W1.0-0. The C engine in `op_oconv.c` and `op_iconv.c` is the authority; the descriptions here follow what it does, not what an upstream manual says it ought to do.

Conversion codes — field 3

A conversion code transforms a value. In a dictionary record, field 3 holds the code the query processor applies to each value before it prints it. In BASIC, the same codes are the second argument to `OCONV()` (for output) and `ICONV()` (for input).

A conversion code is a string whose first character or two identifies the conversion; the rest is parameters. Multiple conversions can be chained with value marks, and each is applied in turn.

DATE CONVERSION — D

`D {y} {c} {fmt}`

A date conversion takes an SD internal date (days since 31 Dec 1967) and formats it for display. The *y* digit controls how many year digits to show; *c* is a calendar code; *fmt* is a format string in brackets.

YEAR DIGITS	WHAT IT SHOWS
(omitted)	four-digit year (default)
2	two-digit year
4	four-digit year

The format string, if present, is enclosed in square brackets and is made of these elements, separated by commas:

ELEMENT	WHAT IT PRODUCES
D	day of month, one or two digits
DD	day of month, two digits
M	month, one or two digits
MM	month, two digits
MA	month name, abbreviated (Jan)
ML	month name, full (January)

ML and the rest of this table are elements of a date mask, used inside a **D** conversion such as **D[DD ML YYYY]**. They are not conversions in their own right: `oconv(20899, 'ML')` returns the value unchanged, because **ML** alone is not a date conversion and SD passes through what it cannot convert. | **Y** | year, two digits | | **YY** | year, four digits | | **J** | Julian day of year | | **Q** | quarter number | | **W** | day of week, abbreviated | | **WL** | day of week, full | | **N** | day of year |

D2[DD/MM/YY] produces `29/08/26`. **D[DD MMM YYYY]** produces `29 Aug 2026`. Without a format string, the default follows the system date format — **D** alone produces `29 Aug 2026` on a machine set to European date order.

TIME CONVERSION — MT

```
MT {H} {S} {c}
```

A time conversion takes an SD internal time (seconds since midnight) and formats it.

OPTION	WHAT IT DOES
H	use 12-hour format with AM/PM suffix
S	include seconds
c	separator character (defaults to colon)

MT produces `14:30`. **MTH** produces `02:30 PM`. **MTHS** produces `02:30:45 PM`.

MASKED DECIMAL — MD, ML, MR

```
MD d {f} {x{c}}
ML d {f} {x{c}}
```

MR d {f} {x{c}}

A masked decimal conversion formats a number with a fixed number of decimal places. MD is the general form; ML left-justifies the result; MR right-justifies it.

PARAMETER	MEANING
d	number of decimal places (0-9)
f	scale factor (0-9), defaults to d
x	field width
c	padding character (defaults to space)

MD2 formats a number with two decimal places: 1234.56. The scale factor matters when the stored value is an integer that represents a scaled number — MD20 with scale factor 0 treats 123456 as 1234.56.

CASE CONVERSION — MC

MCL	MCU	MCC	MCT
-----	-----	-----	-----

CODE	WHAT IT DOES
MCL	convert to lower case
MCU	convert to upper case
MCC	capitalise each word
MCT	title case — the first letter of every word: hello there world becomes Hello There World

RADIX CONVERSION — MX, MO, MCD, MCX

MX	MO	MCD	MCX
----	----	-----	-----

CODE	WHAT IT DOES
MX	decimal to hexadecimal
MO	decimal to octal
MCD	hexadecimal to decimal

CODE	WHAT IT DOES
MCX	decimal to hexadecimal (same as MX)

MX0C converts a hexadecimal string to characters. MB0C and M00C do the same for binary and octal.

BOOLEAN CONVERSION — B

B

Converts a value to Y (true) or N (false). A zero or null is N; anything else is Y. For input conversion, Y becomes 1 and N becomes 0.

LENGTH CONVERSION — L

L{n} {,m}

Returns the length of the string. If n is given, the result is true if the length is n ; if n and m are given, true if between n and m .

PATTERN MATCHING — P

P(xx)

Tests whether the value matches a pattern. The pattern is made of codes: N for numeric, A for alphabetic, X for any character, with a count. P(3N2A) matches three digits followed by two letters.

RANGE TEST — R

Rn,m{;n,m...}

Tests whether the value falls in one or more ranges. R1,10;20,30 is true if the value is between 1 and 10 or between 20 and 30.

SUBSTRING EXTRACTION — T

T{n,}m

Extracts a substring. T3,5 extracts characters 3 through 7. T5 is shorthand for the first 5 characters.

CONCATENATION — C

```
C;v1;v2;v3
```

Concatenates the value with the supplied strings. `C;ABC;` produces the value followed by `ABC`.

SUBSTITUTION — S

```
S;v1;v2;v3
```

Substitutes the value with the n -th string. `S;Y;N` produces `Y` if the value is `1`, `N` if it is `2`.

GROUP CONVERSION — G

```
G{skip}df
```

Splits a string into groups. *skip* is the number of characters to skip between groups; *d* is the delimiter; *f* is the number of characters per group.

BINARY CONVERSIONS — IF, IL, IS

```
IFx      ILx      ISx
```

CODE	WHAT IT DOES
<code>IFx</code>	8 bytes to floating point
<code>ILx</code>	4-byte binary integer to number
<code>ISx</code>	2-byte binary integer to number

x is `L` (low byte first), `H` (high byte first), or omitted for the machine's native byte order.

BASE64 — B64

```
B64
```

For output, encodes the value as Base64. For input, decodes a Base64 string back to binary.

```
Tfile;cv;i;o
```

A translation conversion: looks up the value in *file*, using conversion *cv*, and returns field *i*. If not found, returns *o*. This is the same as the BASIC `TRANS()` function and is how a dictionary field can display data from another file.

FIELD EXTRACTION — ANGLE BRACKETS

```
<f,v,s>
```

Extracts field *f*, value *v*, subvalue *s* from the value treated as a dynamic array. This is the same as the BASIC angle-bracket extraction.

Continued in

SD Dictionaries - Formats and Expressions — format specifications, I-type expressions, correlatives and chaining.